Name: Vanshika Wadhwani
Class: D20A
Roll no: 35

Experiment: 1

Aim: Cryptography in Blockchain, Merkle root Tree Hash.

Tasks Performed:

1. **Hash Generation using SHA-256:**
   - Developed a Python program to compute a SHA-256 hash for any given input string using the `hashlib` library.
2. **Target Hash Generation with Nonce:**
   - Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.
3. **Proof-of-Work Puzzle Solving:**
   - Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.
4. **Merkle Tree Construction:**
   - Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

Program and Output:

Program #1: Python program that uses the hashlib library to create the hash of a given string:

Code:

```
import hashlib

def create_hash(string):
    # Create a hash object using SHA-256 algorithm
    hash_object = hashlib.sha256()
    # Convert the string to bytes and update the hash object
```

```
    hash_object.update(string.encode('utf-8'))
    # Get the hexadecimal representation of the hash
    hash_string = hash_object.hexdigest()
    # Return the hash string
    return hash_string

# Example usage
input_string = input("Enter a string: ")
hash_result = create_hash(input_string)
print("Hash:", hash_result)
```

Output:

```
Enter a string: vanshika
Hash: bedc041e5bac11d84716b08b3794cbf5524c7eec3cb11d0b1de2ad2e0776b1fb
```

Program #2: Program to generate required target hash with input string and nonce

Code:

```
import hashlib

# Get user input
input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")

# Concatenate the string and nonce
hash_string = input_string + nonce

# Calculate the hash using SHA-256
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()

# Print the hash code
print("Hash Code:", hash_code)
```

Output:

```
Enter a string: vanshika wadhwani
Enter the nonce: 2463
Hash Code:
b4220e93d2a2de845a97375f0673e75dbea5d9f0fa806ce6fab1960ef39a2339
```

Program #3: Python code for Solving Puzzle for leading zeros with expected nonce and given string

Code:

```
import hashlib

def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = '0' * num_zeros

    while True:
        # Concatenate the string and nonce
        hash_string = input_string + str(nonce)
        # Calculate the hash using SHA-256
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
        hash_code = hash_object.hexdigest()

        # Check if the hash code has the required number of leading zeros
        if hash_code.startswith(hash_prefix):
            print("Hash:",hash_code )
            return nonce

        nonce += 1

# Get user input
input_string = "Vanshika"
num_zeros = 1

# Find the expected nonce
expected_nonce = find_nonce(input_string, num_zeros)

# Print the expected nonce
print("Input String:", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)
```

Output:

```
Hash: 092dbb615821f809c5914d4eacdc2d799cbbb2f02960923479b4a382c84c14df
Input String: Vanshika
```

```
Leading Zeros: 1
Expected Nonce: 17
```

Program 4: Generating Merkle Tree for given set of Transactions

```
import hashlib

def build_merkle_tree(transactions):
    if len(transactions) == 0:
        return None

    # Hash individual transactions first
    hashed_transactions = []
    print("\n--- Initial Transactions and their Hashes ---")
    for tx in transactions:
        tx_hash = hashlib.sha256(tx.encode('utf-8')).hexdigest()
        hashed_transactions.append(tx_hash)
        print(f"  {tx} -> {tx_hash}")
    transactions = hashed_transactions

    if len(transactions) == 1:
        print(f"\n--- Merkle Tree Construction Step 1 ---")
        print(f"Current layer hashes: {transactions}")
        return transactions[0]

    # Recursive construction of the Merkle Tree
    step = 1
    while len(transactions) > 1:
        print(f"\n--- Merkle Tree Construction Step {step} ---")
        print(f"Current layer hashes: {transactions}")

        if len(transactions) % 2 != 0:
            transactions.append(transactions[-1])
            print(f"  Padded with duplicate: {transactions}")

        new_transactions = []
        for i in range(0, len(transactions), 2):
            hash1 = transactions[i]
            hash2 = transactions[i+1]
            combined = hash1 + hash2
```

```
            hash_combined =
hashlib.sha256(combined.encode('utf-8')).hexdigest()
            print(f"  Combining {hash1[:8]}... and {hash2[:8]}... ->
{hash_combined[:8]}...")
            new_transactions.append(hash_combined)

        transactions = new_transactions
        step += 1

    return transactions[0]

# Example usage
transactions = ["Alice -> Bob : $200", "Bob -> Dave : $500", "Dave -> Eve
: $100", "Eve -> Alice : $300", "Roo -> Bob : $50"]

merkle_root = build_merkle_tree(transactions)
print("\n--- Final Merkle Root ---")
print("Merkle Root:", merkle_root)
```

Output:

```
--- Initial Transactions and their Hashes ---
  Alice -> Bob : $200 ->
f817ce0bddbfa417eba4ae519a8b864ba7b1eed286e10c4b92086cc713279760
  Bob -> Dave : $500 ->
768890d3b58d4d6a17673e6f750a0145f546557c9529d258750e1ca0cdbb5b46
  Dave -> Eve : $100 ->
43dd5729f00e01eea73a858d02f9c714c882bab801de203c1314d6318d6890da
  Eve -> Alice : $300 ->
cba341d7965fff73181a2a6579799dc9644e84380fa9e14f12ebf78c70316a8a
  Roo -> Bob : $50 ->
cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac

--- Merkle Tree Construction Step 1 ---
Current layer hashes:
['f817ce0bddbfa417eba4ae519a8b864ba7b1eed286e10c4b92086cc713279760',
'768890d3b58d4d6a17673e6f750a0145f546557c9529d258750e1ca0cdbb5b46',
'43dd5729f00e01eea73a858d02f9c714c882bab801de203c1314d6318d6890da',
```

```
'cba341d7965fff73181a2a6579799dc9644e84380fa9e14f12ebf78c70316a8a',
'cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac']
  Padded with duplicate:
['f817ce0bddbfa417eba4ae519a8b864ba7b1eed286e10c4b92086cc713279760',
'768890d3b58d4d6a17673e6f750a0145f546557c9529d258750e1ca0cdbb5b46',
'43dd5729f00e01eea73a858d02f9c714c882bab801de203c1314d6318d6890da',
'cba341d7965fff73181a2a6579799dc9644e84380fa9e14f12ebf78c70316a8a',
'cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac',
'cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac']
  Combining f817ce0b... and 768890d3... -> 471608d3...
  Combining 43dd5729... and cba341d7... -> a61c504d...
  Combining cfe1dc26... and cfe1dc26... -> 314ca746...

--- Merkle Tree Construction Step 2 ---
Current layer hashes:
['471608d3d6f2a642ff3a84dfaeeaebb1c271a9aed5b37a82d61c4784558e80e9',
'a61c504d72e6ce69b2ec8791ff7469cc004704f727f8dbbf420fadbc4e4c0de7',
'314ca746b3dafe63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e']
  Padded with duplicate:
['471608d3d6f2a642ff3a84dfaeeaebb1c271a9aed5b37a82d61c4784558e80e9',
'a61c504d72e6ce69b2ec8791ff7469cc004704f727f8dbbf420fadbc4e4c0de7',
'314ca746b3dafe63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e',
'314ca746b3dafe63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e']
  Combining 471608d3... and a61c504d... -> ae847393...
  Combining 314ca746... and 314ca746... -> 6bb2c8b4...

--- Merkle Tree Construction Step 3 ---
Current layer hashes:
['ae847393c3dff60e1d95cfdbb1c1c886d0bf7cbd26410bcc2469e023e94cb8ee',
'6bb2c8b4fd23658c0b716069761d9811ced49f7e4f2f736b493fe0c0a96b4f45']
  Combining ae847393... and 6bb2c8b4... -> 96ee90df...

--- Final Merkle Root —
Merkle Root:
96ee90df2272d24c0bcf4c67e152c257dc45bce4560221125b0750cead7b27b0
```

Conclusion:

This study demonstrates fundamental cryptographic concepts and their applications within blockchain technology:

- **SHA-256 Hashing** provides a secure and unique fingerprint for any input, ensuring data integrity.
- **Nonce and Target Hashing** allow the simulation of proof-of-work consensus, emphasizing the difficulty of mining blocks by meeting hash conditions (leading zeros).
- **Proof-of-Work Puzzle Solving** shows how mining requires computational effort to find a nonce making the hash satisfy network difficulty.
- **Merkle Tree Construction** efficiently summarizes and validates large numbers of transactions with a single Merkle root hash, which is essential in blockchain for fast verification and ensuring the integrity of data blocks.

Together, these implementations capture the essence of how cryptography underpins blockchain's security, immutability, and efficiency, especially through hash functions and structures like Merkle Trees that enable trustless and decentralized transaction management.